ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 34

Arquitectura interna de Laravel

- Los servicios: código que no es ni controlador ni modelo
- El contenedor de servicios
- Service providers
- Contratos: programar contra una interfaz
- Facades
- Middleware
- El ciclo de vida, ahora sí completo
- Nuestro MVC frente a Laravel



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Llevamos siete clases **usando** Laravel. Hemos pedido un `Request` como parámetro y ha aparecido; hemos escrito `Auth::user()` o `DB::table()` sin crear ningún objeto; hemos puesto `->middleware('auth')` y la ruta se ha protegido sola. Hoy levantamos el capó para ver **cómo** pasa todo eso.

Y la buena noticia es que ya lo sabéis. En la clase 25 construimos un `Contenedor` con *autowiring*, un fichero `config/servicios.php` donde registrábamos los servicios y una cadena de middleware. Hoy vemos las mismas piezas en Laravel y, sobre todo, aprendemos a **poner nuestro código en ellas**.

•
👌 Cómo vamos a trabajar



1
📘 Los servicios: código que no es ni controlador ni modelo



2
😮 El contenedor de servicios



3
🚀 Service providers



4
🤔 Contratos: programar contra una interfaz



5
🔍 Facades



6
📘 Middleware



7
👌 El ciclo de vida, ahora sí completo



8
😮 Nuestro MVC frente a Laravel



9
🚀 Ejercicios



10
👋 Lo que tenéis que llevaros de hoy





CLASE 34

👌 Cómo vamos a trabajar

Copiad el contenido de `ejemplos/` encima del proyecto. No hay migraciones nuevas.

Esquema

```
app/Servicios/CalculadoraPrecios.php    ← NUEVO: un servicio
app/Contracts/Notificador.php           ← NUEVO: un contrato (interfaz)
app/Servicios/NotificadorLog.php        ← NUEVO: una implementación del contrato
app/Facades/Precios.php                 ← NUEVO: nuestra propia fachada
app/Providers/AcademiaServiceProvider.php ← NUEVO: registra todo lo anterior
app/Http/Middleware/                    ← NUEVOS: ComprobarRol y CabecerasSeguridad
bootstrap/app.php y bootstrap/providers.php ← registran el middleware y el provider
config/academia.php                     ← IVA y descuento máximo
CursoController, PaginaController, Curso y varias vistas
routes/web.php                          ← presupuesto y /admin/sistema (la antigua /info)
```



💡 Si os da algún error raro de clases que no se encuentran después de copiar, ejecutad `composer dump-autoload` y `php artisan optimize:clear`.



CLASE 34 · PUNTO 1

📘 Los servicios: código que no es ni controlador ni modelo

Hasta ahora, cada línea de código tenía un sitio claro: las consultas en el modelo, la lógica de la petición en el controlador, el HTML en la vista. Pero ¿dónde va el cálculo de un presupuesto con IVA y descuentos? Se usa en la web, en un comando de consola y, mañana, en la API. Si lo metemos en un controlador, los demás no pueden usarlo.

La respuesta es un **servicio**: una clase normal, de las de la clase 16, que hace **una sola tarea** (`CalculadoraPrecios.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
namespace App\Servicios;

// Un servicio: una clase que hace UNA tarea y que no es ni controlador ni modelo
class CalculadoraPrecios
{
    public function __construct(
        private float $iva,
        private int $descuentoMaximo,
    ) {
    }

    public function conIva(float $precio): float
    {
        return round($precio * (1 + $this->iva), 2);
    }

    public function conDescuento(float $precio, int $porcentaje): float
    {
        // Nunca más descuento del permitido por la academia
        $porcentaje = min($porcentaje, $this->descuentoMaximo);

        return round($precio * (100 - $porcentaje) / 100, 2);
    }

    public function formatear(float $importe): string
    {
        return number_format($importe, 2, ',', '.') . ' €';
    }
}
```

La carpeta `app/Servicios` no existe en Laravel: la creamos nosotros. Laravel no impone dónde poner estas clases; basta con que el *namespace* coincida con la carpeta para que Composer las encuentre (clase 19).



💡 Fijaos en que el IVA y el descuento máximo **no** están escritos dentro de la clase: los recibe en el constructor. Así es fácil cambiarlos (en `config/academia.php`) y probar la clase con otros valores (clase 36).



CLASE 34 · PUNTO 2

😮 El contenedor de servicios

El **contenedor de servicios** (*service container*) es el corazón de Laravel: el objeto que **sabe crear** todos los demás y se los da a quien los pida [1]. Es exactamente nuestro `Contenedor` de la clase 25.

Inyección automática (autowiring)

Cuando un controlador pide algo con su tipo, el contenedor lo crea y se lo pasa (`CursoController.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// GET /cursos/{curso}/presupuesto?descuento=10
// El contenedor crea la calculadora (con su IVA) y nos la pasa: inyección de dependencias
public function presupuesto(Request $request, Curso $curso, CalculadoraPrecios
$calculadora): View
{
    $descuento = (int) $request->query('descuento', 0);
    $conDescuento = $calculadora->conDescuento((float) $curso->precio, $descuento);

    return view('cursos.presupuesto', [
        'curso' => $curso,
        'descuento' => $descuento,
        'calculadora' => $calculadora,
        'base' => $conDescuento,
        'total' => $calculadora->conIva($conDescuento),
    ]);
}
```

Tres parámetros y tres orígenes distintos: el `Request` lo pone Laravel, el `Curso` sale del *route model binding* y la `CalculadoraPrecios` , del contenedor. Nosotros no hemos escrito ni un `new` .

Si el constructor de una clase solo pide otras clases, el contenedor las crea él solo leyendo los tipos con *reflection* (como hacía el nuestro). Pero `CalculadoraPrecios` pide un `float` y un `int` : ¿cuánto vale el IVA? Eso el contenedor no lo puede adivinar, así que **se lo enseñamos**.

Registrar un servicio

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// El mismo objeto para toda la petición
$this->app->singleton(CalculadoraPrecios::class, function () {
    return new CalculadoraPrecios(
        iva: config('academia.iva'),
        descuentoMaximo: config('academia.descuento_maximo'),
    );
});

// Un objeto nuevo cada vez que se pida
$this->app->bind(Notificador::class, NotificadorLog::class);
```

Método	Qué hace
`bind(Clase::class, ...)`	Cada vez que se pida, crea un objeto **nuevo**
`singleton(Clase::class, ...)`	Lo crea la **primera** vez y después devuelve siempre el mismo
`instance(Clase::class, $objeto)`	Guarda un objeto ya creado
`app(Clase::class)` o `app()->make(Clase::class)`	Pide un objeto al contenedor desde cualquier sitio

¿Y dónde se escriben estos registros? En un *service provider*.



Los ***service providers*** son las clases que **preparan la aplicación** al arrancar: registran servicios, configuran cosas y preparan las vistas. Son nuestro `config/servicios.php` de la clase 25 [2]. Laravel trae uno, `AppServiceProvider`, y nosotros creamos otro para la academia:

terminal — php-paso-a-paso/clase34 — Visual Studio Code

terminal

```
php artisan make:provider AcademiaServiceProvider
```

Artisan lo añade a la lista de `bootstrap/providers.php`, que es la que Laravel recorre al arrancar:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
return [
    AppServiceProvider::class,
    AcademiaServiceProvider::class,
];
```

Cada *provider* tiene dos métodos (`AcademiaServiceProvider.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
class AcademiaServiceProvider extends ServiceProvider
{
    // register(): SOLO enseñar al contenedor cómo crear cada servicio
    public function register(): void
    {
        // singleton: el mismo objeto para toda la petición
        $this->app->singleton(CalculadoraPrecios::class, function () {
            return new CalculadoraPrecios(
                iva: config('academia.iva'),
                descuentoMaximo: config('academia.descuento_maximo'),
            );
        });

        // Cuando alguien pida el contrato Notificador, le damos un NotificadorLog
        $this->app->bind(Notificador::class, NotificadorLog::class);
    }

    // boot(): lo que necesita que TODO esté ya registrado
    public function boot(): void
    {
        // Un "view composer": cada vez que se pinte el menú, recibe las categorías
        View::composer('partials.menu', function ($vista) {
            $vista->with('categoriasMenu', Categoria::orderBy('nombre')->get());
        });
    }
}
```

¿Por qué dos métodos? Porque Laravel primero llama al `register()` de **todos** los *providers* y después al `boot()` de todos. En `register()` aún no podemos usar otros servicios (puede que no estén registrados); en `boot()`, sí. Es lo que hicimos en las clases 31 y 33 con `preventLazyLoading()`, `Paginator` y las *gates*: todo en `boot()`.

Los view composers

El menú necesita las categorías **en todas las páginas**. Pasarlas desde cada controlador sería un rollo y nos olvidaríamos en alguno. Un **view composer** es una función que se ejecuta **cada vez que se pinta una vista** y le añade datos. Ahora el menú tiene un enlace por categoría:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```blade
{{-- $categoriasMenu llega del view composer de AcademiaServiceProvider --}}
@foreach ($categoriasMenu as $categoria)
    <a href="{{ route('categorias.show', $categoria) }}">{{ $categoria->nombre }}</a>
@endforeach
```



CLASE 34 · PUNTO 4

🤔 Contratos: programar contra una interfaz

Cuando alguien escribe en el formulario de contacto, queremos avisar a la academia. ¿Cómo? Hoy, apuntándolo en el log; en la clase 37, por correo; quizá mañana, por Telegram. Si el controlador usara directamente una de esas clases, cambiar de una a otra obligaría a tocar el controlador.

La solución es la de la clase 17: una **interfaz**. En Laravel las llaman **contratos** (`Notificador.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
namespace App\Contracts;

// Un contrato: QUÉ se puede hacer, sin decir CÓMO
interface Notificador
{
    public function notificar(string $destinatario, string $asunto, string $mensaje): void;
}
```

Una implementación (`NotificadorLog.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
class NotificadorLog implements Notificador
{
    public function notificar(string $destinatario, string $asunto, string $mensaje): void
    {
        Log::info("Notificación para {$destinatario}: {$asunto}", ['mensaje' => $mensaje]);
    }
}
```

El controlador pide **la interfaz** (`PaginaController.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// El Notificador lo inyecta el contenedor: el controlador no sabe qué clase es
public function enviarContacto(Request $request, Notificador $notificador): RedirectResponse
{
    $datos = $request->validate([...]);

    $notificador->notificar(
        config('academia.email'),
        "Nuevo mensaje de {$datos['nombre']} ({$datos['email']})",
        $datos['mensaje'],
    );
    ...
}
```

Y el *provider* decide qué clase se usa: `$this->app->bind(Notificador::class, NotificadorLog::class)`. El día que queramos correo, cambiamos **esa línea**. Enviad el formulario y mirad el final de `storage/logs/laravel.log`:

Esquema

```
[2026-11-07 21:53:26] local.INFO: Notificación para info@academia-alpe.test: Nuevo mensaje de Ana (ana@example.com) {"mensaje":"Quiero información del curso"}
```

Es el principio de **inversión de dependencias**, la «D» de SOLID que vimos en la clase 21: depender de abstracciones, no de clases concretas.

Laravel usa la misma idea por dentro: en `Illuminate\Contracts` hay una interfaz para cada servicio (caché, correo, colas…). Por eso podéis cambiar en el `.env` la caché de la base de datos a Redis sin tocar una línea de código: vuestro código usa el contrato, y la configuración elige la implementación.



Llevamos semanas escribiendo `Route::get()`, `DB::table()`, `Auth::user()`, `Storage::disk()`… Parecen métodos **estáticos**, pero no lo son. Son ***facades*** («fachadas»): clases que hacen de **atajo** a un objeto del contenedor [3].

Cuando escribimos `Auth::user()`, por dentro pasa esto:

Esquema

```
Auth::user()
   │  la clase Auth (fachada) dice: «yo represento al servicio "auth"»
   ▼
app('auth')          ← pide el objeto al contenedor
   │
   ▼
app('auth')->user()  ← y le pasa la llamada
```

Para entenderlo, hemos creado **nuestra propia fachada** para la calculadora (`app/Facades/Precios.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
namespace App\Facades;

use App\Servicios\CalculadoraPrecios;
use Illuminate\Support\Facades\Facade;

class Precios extends Facade
{
    protected static function getFacadeAccessor(): string
    {
        return CalculadoraPrecios::class;
    }
}
```

¡Ya está! Solo dice **qué objeto del contenedor representa**. Ahora el *accessor* del modelo `Curso` puede usarla sin inyectar nada (`Curso.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
protected function precioConIva(): Attribute
{
    return Attribute::make(
        // La fachada Precios usa la CalculadoraPrecios del contenedor
        get: fn () => Precios::conIva((float) $this->precio),
    );
}
```

Y el IVA ya solo está en **un sitio**: `config/academia.php`.

¿Fachada, función o inyección?

Las tres llegan al mismo objeto. Usad la que haga el código más claro:

Fachada	Función (*helper*)	Inyección
`Auth::user()`	`auth()->user()`	`Request $request` → `$request->user()`
`View::make('inicio')`	`view('inicio')`	
`Config::get('app.name')`	`config('app.name')`	
`Cache::get('clave')`	`cache('clave')`	`Illuminate\Contracts\Cache\Repository $cache`
`Precios::conIva(100)`		`CalculadoraPrecios $calculadora`



💡 **Mi consejo:** en los controladores y servicios, preferid la **inyección**: con solo leer el constructor o los parámetros, sabéis de qué depende la clase, y es más fácil de probar. Las fachadas y los *helpers*, para las vistas, las rutas y el código corto.



CLASE 34 · PUNTO 6

📘 Middleware

Ya hemos usado `auth`, `guest`, `throttle` y `can`. Un middleware es una **capa** por la que pasa la petición antes de llegar al controlador, y la respuesta al volver [4]:

Esquema

```
Petición ──► [CabecerasSeguridad] ──► [auth] ──► [rol:admin] ──► Controlador
Respuesta ◄── [CabecerasSeguridad] ◄── [auth] ◄── [rol:admin] ◄──┘
```

terminal — php-paso-a-paso/clase34 — Visual Studio Code

terminal

```
php artisan make:middleware ComprobarRol
```

Un middleware con parámetros

`ComprobarRol.php` comprueba el rol **antes** de dejar pasar:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// Uso en las rutas: ->middleware('rol:admin')  o  ->middleware('rol:admin,editor')
class ComprobarRol
{
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $usuario = $request->user();

        if ($usuario === null || ! in_array($usuario->rol->value, $roles, true)) {
            abort(403);
        }

        // Todo en orden: que la petición siga hacia el controlador
        return $next($request);
    }
}
```

- `$next($request)` pasa la petición a la siguiente capa. Si no lo llamamos, la petición **se queda aquí**.
- Lo que va después de los dos puntos en `rol:admin,editor` llega como parámetros extra del `handle()`.

Un middleware que actúa después

`CabecerasSeguridad.php` deja pasar la petición y **retoca la respuesta**:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
public function handle(Request $request, Closure $next): Response
{
    $respuesta = $next($request);

    // Que nadie pueda meter nuestra web en un <iframe> (clickjacking)
    $respuesta->headers->set('X-Frame-Options', 'SAMEORIGIN');
    // Que el navegador no intente adivinar el tipo de los archivos
    $respuesta->headers->set('X-Content-Type-Options', 'nosniff');

    return $respuesta;
}
```

Registrarlos

En Laravel 13, el middleware se configura en `bootstrap/app.php`:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
->withMiddleware(function (Middleware $middleware): void {
    // Para todas las peticiones
    $middleware->append(CabecerasSeguridad::class);

    // Un nombre corto para usarlo en las rutas
    $middleware->alias([
        'rol' => ComprobarRol::class,
    ]);
})
```

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// routes/web.php: la antigua página /info de la clase 27, ahora solo para el administrador
Route::get('/admin/sistema', [PaginaController::class, 'sistema'])->middleware('rol:admin')
->name('admin.sistema');
```

Forma	Se aplica a...
`$middleware->append(...)`	Todas las peticiones (web y API)
`$middleware->appendToGroup('web', ...)`	Solo las rutas de `routes/web.php`
`$middleware->alias([...])` + `->middleware('nombre')`	Solo las rutas que lo pidan



💡 ¿ `rol:admin` o `can:ver-panel` ? Hacen cosas parecidas. El middleware de rol es más directo; las *gates* y *policies* son más flexibles (pueden mirar el modelo concreto). En proyectos grandes se suele preferir la autorización con *policies*; aquí tenéis las dos para que sepáis leerlas.



CLASE 34 · PUNTO 7

👌 El ciclo de vida, ahora sí completo

En la clase 27 vimos el viaje de una petición a vista de pájaro. Con lo de hoy, podemos ponerle nombre a cada parada [5]:

Esquema

```
 1. public/index.php          Carga el autoload de Composer y bootstrap/app.php
 2. bootstrap/app.php         Crea la aplicación (que ES el contenedor) y configura
                              rutas, middleware y errores
 3. Service providers         register() de todos (bootstrap/providers.php y los de los
paquetes)
                              y después boot() de todos
 4. Kernel HTTP               Convierte la petición en un objeto Request
 5. Middleware global         CabecerasSeguridad, recorte de espacios, mantenimiento…
 6. Router                    Busca la ruta y aplica su middleware (web, auth, rol…)
 7. Controlador               El contenedor le inyecta lo que pida
 8. Response                  Vuelve por todas las capas de middleware al revés
 9. Navegador                 Y Laravel termina (las tareas «para después», si las hay)
```



Cerramos el círculo que abrimos en la clase 25. Esta es la tabla completa, ahora que ya habéis usado cada pieza de Laravel:

Nuestro MVC	Laravel	Clase
`public/index.php`	`public/index.php`	27
`Aplicacion`	`Illuminate\Foundation\Application` (el contenedor)	27 y 34
`.env` + `Entorno`	`.env` + `env()` en `config/` y `config()` en el código	27
`config/rutas.php` + `Router`	`routes/web.php` + `Route`	28
`Peticion` / `Respuesta`	`Request` / `Response`	28
`Controlador` base	`Controller`	28
`HttpException` / `noEncontrado()`	`abort(404)`	28
`Vista` + layout + `parcial()`	Blade: `@extends`, `@include` y componentes	29
`base-de-datos.sql`	Migraciones y *seeders*	30
`BaseDatos` (PDO)	`DB` y el Query Builder	30
`Modelo` (Active Record)	Eloquent	31
`$peticion->validar([...])`	`$request->validate([...])` y *Form Requests*	32
`Sesion::flash()`	`->with()` y `@session`	32
`campoCsrf()` + `VerificarCsrf`	`@csrf` + middleware CSRF	32
`Auth` + `Autenticado`	`Auth` + middleware `auth`	33
`Contenedor` con *autowiring*	*Service container*	34
`config/servicios.php`	*Service providers*	34
`Middleware` + `grupo()`	Middleware + `Route::middleware()->group()`	34
`ManejadorErrores`	`withExceptions()` en `bootstrap/app.php`	37
API con `Respuesta::json()` y token	API Resources y Sanctum	35

Si os paráis a pensarlo, es impresionante: habéis construido a mano una versión pequeña de **cada pieza** de un *framework* profesional. Por eso Laravel ya no es magia para vosotros. 💪



Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Registrar las visitas.** Cread el middleware `RegistrarVisitas`, que apunte en el log cada petición de la web con su verbo, la dirección, el código de estado, los milisegundos que ha tardado y el correo

del usuario (o «invitado»). Registradlo solo para el grupo `web`. → `RegistrarVisitas.php` y `bootstrap/app.php`

- ☐ **02 · Otro notificador.** Cread `NotificadorArchivo`, que implemente el contrato `Notificador` y añada cada notificación como una línea de `storage/app/private/notificaciones.txt` (con `Storage::append()`). Haced que el *provider* elija la implementación según la variable `ACADEMIA_NOTIFICADOR` del `.env` (`log` o `archivo`). → `NotificadorArchivo.php`, `AcademiaServiceProvider.php` y `config/academia.php`
- ☐ **03 · Presupuesto por consola.** Cread el comando `php artisan academia:presupuesto {curso} {--descuento=0}`, que reciba la `CalculadoraPrecios` como parámetro de la función (los comandos también tienen inyección) y muestre una tabla con el precio, el precio con descuento y el total con IVA. → `routes/console.php`



🏆 **Reto:** cread un servicio `EstadisticasAcademia` con los métodos `totalCursos()`, `totalAlumnos()`, `precioMedio()` y `cursoMasPopular()`, registradlo como *singleton* y usadlo tanto en el comando `academia:estadisticas` de la clase 30 como en el panel de administración. Una sola clase, dos sitios.



CLASE 34 · PUNTO 10

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Servicio	Una clase que hace una tarea y se reutiliza en web, consola y API
Contenedor	El objeto que sabe crear todos los demás y los inyecta
`bind` / `singleton`	Objeto nuevo cada vez / siempre el mismo
Service provider	Prepara la aplicación: `register()` registra, `boot()` configura
`bootstrap/providers.php`	La lista de nuestros *providers*
View composer	Datos para una vista cada vez que se pinta (el menú)
Contrato	Una interfaz: el código depende del QUÉ, la configuración elige el CÓMO
Facade	Un atajo con aspecto estático a un objeto del contenedor
Middleware	Capas antes y después del controlador; `$next($request)` sigue el camino
`bootstrap/app.php`	Donde se registra el middleware (`append`, `alias`…)

📚 Fuentes y documentación oficial

1. Laravel: Service Container
2. Laravel: Service Providers
3. Laravel: Facades
4. Laravel: Middleware
5. Laravel: Request Lifecycle
6. Laravel: Contracts
7. Laravel: Views, View Composers